Coding Practice Hub

Antigravity Master Prompt - Complete Local Daily Coding Practice Application

Goal: Build a complete, maintainable local web application for daily technical coding and interview practice. The application is a personal learning system used every day for SQL, Python, PySpark, Databricks, DSA, and System Design.

```
Daily target: 6 practices / day
1 SQL | 1 Python | 1 PySpark | 1 Databricks | 1 DSA | 1 System Design
```

The application must create daily practice sessions, execute supported code, save code and output, preserve successful and failed attempts, allow editing of historical practice, manage datasets, and provide dashboards and reports.

1. Technology Stack

Backend

• Python 3.12.10

• Flask

• SQLite for application metadata/history

• DuckDB for SQL practice

• Local Python interpreter for Python execution

• Local PySpark/Spark for PySpark execution

Frontend

• Flask templates / Jinja2

• HTML5

• CSS3

• Vanilla JavaScript

• Monaco Editor

Do not use React, Angular, Vue, Next.js, or a separate frontend server. Flask must serve both frontend and backend through one port.

```
python app.py

Application: http://localhost:5000
```

2. Daily Practice Model

Every calendar day is one Daily Practice Session containing exactly six technology practice records. Do not model them as six unrelated sessions.

```
Daily Practice Session
|- SQL
|- Python
|- PySpark
|- Databricks
|- DSA
`- System Design

Daily target: 6 / 6 completed
```

3. Main Navigation

```
Dashboard | Create Practice | Practice History | Reports | Datasets | Settings
```

Use a professional developer-tool appearance, responsive but primarily optimized for desktop/laptop use.

4. Dashboard

• Top cards: Total Practices, Successful, Failed, In Progress, Not Started, Total Executions, Current Streak, Longest Streak

• Technology statistics for SQL, Python, PySpark, Databricks, DSA, and System Design

• Today's progress with individual statuses and overall x/6 completion

• Start Today's Practice or Continue Today's Practice based on session existence

```
Technology       Total    Success    Failed    In Progress
SQL
Python
PySpark
Databricks
DSA
System Design
```

5. Dashboard Filters and Grouping

• Multi-select filters: Date, Date Range, Technology, Status, Difficulty, Topic, Tags

• Allow combined filters such as PySpark + Failed + Last 30 Days

• Group By: Date, Technology, Status, Difficulty, Topic

• Dashboard/report data must update based on selected filters

6. Create Practice

Create or open today's Daily Practice Session. Enforce one session per date and prevent accidental duplicates.

```
SQL | Python | PySpark | Databricks | DSA | System Design

Statuses: NOT STARTED | IN PROGRESS | RUNNING | SUCCESS | FAILED | COMPLETED
```

7. Practice Workspace Layout

For SQL, Python, PySpark, Databricks and DSA, use approximately 70% Editor and 30% Output. Make the divider resizable where practical. Use Monaco Editor with syntax highlighting, line numbers, search, formatting where possible, keyboard shortcuts, theme support, and automatic indentation.

```
+-------------------------------------+------------------+
|             CODE EDITOR 70%         |    OUTPUT 30%    |
|                                     |                  |
+-------------------------------------+------------------+
```

8. Practice Metadata

• Title

• Technology

• Practice Date

• Question / Problem Statement

• Difficulty: Easy / Medium / Hard

• Topic

• Tags

• Notes

• Status

• Created At

• Updated At

• Completed At

9. Execution Workflow

• Buttons: Run, Save, Save & Complete, Clear Output, Reset, Execution History

• On Run: save current code, set RUNNING, execute, capture stdout/output and stderr/error, calculate duration, determine SUCCESS/FAILED, persist execution, display result

• Never discard failed executions

10. Execution History

Every execution must be stored separately. Preserve the exact code snapshot and result for each attempt.

```
Execution #1  FAILED
Execution #2  FAILED
Execution #3  SUCCESS
Execution #4  SUCCESS
```

• Store: Execution ID, Practice ID, Code Snapshot, Output, Error Output, Status, Start/End Time, Duration, Created At

• View historical code/output/error/status/time

• Actions: View, Restore Code, Copy Code

• Restoring old code must not delete newer history

11. Autosave

• Autosave code and notes periodically

• Save when changing technology tabs

• Save before execution

• Save when navigating away where practical

• Display Saving... and Saved states

• Protect against accidental browser closure

12. SQL Practice

Use DuckDB as the SQL practice engine. Provide SQL Editor, database/table explorer, tabular output/results, execution time and rows returned.

• Support SELECT, JOIN, GROUP BY, window functions, CTEs, subqueries, aggregations, DDL, DML and analytical SQL

• Display errors clearly

• Uploaded CSV/JSON/Parquet data should be easy to query

13. Python Practice

• Execute Python locally

• Capture stdout, stderr, exceptions and duration

• Store both code and output

• Prefer a separate subprocess/execution layer rather than executing user code inside the Flask server process

• Use timeout handling so infinite loops or broken programs cannot crash/freeze Flask

14. PySpark Practice

• Support local PySpark

• DataFrames, joins, aggregations, window functions, transformations, actions, Spark SQL

• Read CSV, JSON and Parquet

• Capture Spark output/errors and duration

• A failed Spark exercise must not crash Flask

15. Databricks Practice

Databricks must be its own technology category, separate from PySpark.

```
Subtypes:
PySpark | Python | SQL | Delta Lake | Structured Streaming
Auto Loader | Unity Catalog | Performance / Optimization
Notebook Exercise | Other
```

For V1, Databricks may primarily be a local code/notebook-style practice and knowledge workspace. Actual Databricks workspace connectivity is optional. Store code, expected output, actual/local output where executable, notes, questions and solutions. Design the data/API model so cloud execution can be added later without redesigning the application.

16. DSA Practice

• Fields: Problem Title, Problem Statement, Difficulty, Topic, Constraints, Examples, Code, Output, Test Cases, Notes, Time Complexity, Space Complexity

• Topics: Arrays, Strings, Hash Maps, Linked Lists, Stacks, Queues, Trees, Graphs, Heaps, Recursion, Backtracking, Binary Search, Sorting, Greedy, Dynamic Programming, Sliding Window, Two Pointers, Intervals, Tries, Other

• Use Python execution and preserve every attempt

• Statuses can include Solved, Needs Review, Failed, In Progress

17. System Design

Do not use a normal code interpreter. Create a structured workspace with Markdown and, if practical, Mermaid diagrams.

• Problem

• Requirements

• Functional Requirements

• Non-Functional Requirements

• Capacity Estimates

• API Design

• Database Design

• High-Level Architecture

• Caching

• Messaging

• Scaling

• Reliability

• Security

• Trade-offs

• Bottlenecks

• Notes

• Final Solution

Use approximately 70% Design Workspace and 30% Preview / Notes.

18. Dataset Management

• Upload CSV, JSON, Parquet

• Store files locally

• Show File Name, File Type, File Size, Uploaded Date, Rows, Columns

• Actions: Preview, Use in SQL, Use in Python, Use in PySpark, Delete

• Make datasets queryable through DuckDB and safely expose local paths to Python/PySpark

19. Practice History

Date	SQL	Python	PySpark	Databricks	DSA	Design
12-Oct-26	OK	OK	IP	NS	NS	NS
11-Oct-26	OK	OK	OK	OK	OK	OK

• Click a date to open the exact Daily Practice Session

• Click a technology to open that saved practice

• Actions: View, Edit, Continue

• Edit must restore question, code, latest output, notes, status, metadata and execution history

• Never overwrite historical attempts

20. Technology Views

Provide browse views for each technology with Date, Title, Status, Difficulty and Runs, plus filtering/search and View/Edit actions.

21. Global Search

• Search Titles, Questions, Code, Notes, Tags, Topics and Technology

• A search such as "window function" should find matching SQL, PySpark, Databricks, DSA or other records

22. Reports

Overall: Total Practices, Successful, Failed, In Progress, Completion Rate, Success Rate.

By technology: Total, Success, Failed, In Progress, Completion %, Success %.

Trends: Practices per day/week/month. **Consistency:** Current Streak, Longest Streak, Days Completed 6/6, Missed Days, Partial Days. **Difficulty:** Easy/Medium/Hard. **Execution:** Total, successful, failed, average attempts before success, average execution time.

23. Daily Completion Logic

A day is 6/6 COMPLETE only when all six technology practices are completed. Support and visually display every state from 0/6 through 6/6.

24. Database Design

Use normalized SQLite application tables similar to:

```
daily_sessions
practices
executions
datasets
tags
practice_tags
settings
```

Use primary keys, foreign keys, unique constraints and useful indexes. Enforce one Daily Practice Session per calendar date and prevent duplicate technology records within the same session unless intentionally supported.

```
daily_session
  |- SQL practice -> executions...
  |- Python practice -> executions...
  |- PySpark practice -> executions...
  |- Databricks practice -> executions...
  |- DSA practice -> executions...
  `- System Design practice
```

25. Suggested Project Structure

```
coding-practice-hub/
|- app.py
|- config.py
|- requirements.txt
|- README.md
|- database/
|  |- app.db
|  `- practice.duckdb
|- routes/
|- services/
|- models/
|- templates/
|- static/
|  |- css/
|  |- js/
|  `- vendor/monaco/
|- uploads/
|  |- csv/
|  |- json/
|  `- parquet/
|- workspace/
|  |- python/
|  |- sql/
|  |- pyspark/
|  |- databricks/
|  `- dsa/
|- logs/
`- backups/
```

Do not put everything in app.py. Maintain clear separation of concerns.

26. UI / UX

• Clean modern dashboard and developer-tool appearance

• Sidebar/top navigation

• Compact status cards and readable tables

• Responsive filters

• Clear success/failed/in-progress badges

• Dark/light theme

• Resizable editor/output panels where practical

• Confirmation dialogs for destructive actions

• Do not make the application look like a toy/demo

27. Error Handling

• Handle Python execution failure, PySpark/Java unavailable, DuckDB query errors, missing datasets, invalid files, database errors and timeouts gracefully

• Show useful errors in the Output panel

• User code failure must never crash the Flask application

28. Local Code Execution Safety

• Use subprocesses where appropriate

• Execution timeout and process termination

• Capture stdout/stderr

• Temporary execution workspace

• File path validation

• Prevent infinite Python/PySpark scripts from permanently freezing Flask

29. Settings

• Python executable

• JAVA_HOME

• SPARK_HOME

• PySpark configuration

• Execution timeout

• Upload directory

• Workspace directory

• DuckDB database

• Theme

• Autosave interval

• Automatically detect local Python/Java/Spark configuration where practical

30. Backup and Restore

Provide Create Backup and Restore Backup with confirmation. Back up SQLite, DuckDB where necessary, uploaded datasets, workspace files and configuration so months/years of practice history are protected.

31. First Launch Initialization

- Detect whether databases exist
- Initialize database if missing
- Create required directories
- Create tables/indexes automatically
- Start Flask and serve normally
- Do not require manual database table creation

32. README / Windows Setup

Generate a proper README with Python 3.12.10, virtual environment, pip install, Java/PySpark requirements, DuckDB, Flask, database initialization, startup, troubleshooting and backup instructions.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py

Open: http://localhost:5000
```

33. Highest-Priority Working Flow

```
Launch Application
  -> Dashboard
  -> Create Today's Practice
  -> Six technology tabs
  -> Enter Question
  -> Write Code
  -> Run
  -> SUCCESS / FAILED
  -> Store Code + Output/Error + Execution History
  -> Save Practice
  -> View from History
  -> Edit and restore saved content
  -> Run again
  -> Add new execution without destroying old history
  -> Dashboard/Reports update automatically
```

Do not generate only UI mockups. Implement this workflow end-to-end.

34. Development Approach

```
Phase 1: Project structure, Flask, config, SQLite schema, base UI/navigation
Phase 2: Daily Session, Create Practice, six tabs, save/edit/autosave
Phase 3: Monaco, Python executor, DuckDB SQL executor, output, history
Phase 4: PySpark execution, dataset upload/preview
Phase 5: DSA, Databricks workspace, System Design workspace
Phase 6: Dashboard, filters, reports, grouping, streaks
Phase 7: Backup/restore, settings, themes, UI polish, tests
```

Verify existing functionality after every phase. Do not replace working features with placeholders later.

35. Testing

- Creating a daily session
- Preventing duplicate sessions
- Creating six practice records
- Saving/updating code
- Execution history
- Successful and failed Python execution
- Successful and failed SQL execution
- Report counts
- Filtering
- Daily 6/6 completion calculation

36. Final Acceptance Criteria

1. Start everything from the Flask application
2. Access everything through one port
3. Create one daily session
4. Practice all six technologies using tabs
5. Write SQL/Python/PySpark/DSA/Databricks code
6. Execute supported code
7. See SUCCESS or FAILED
8. See output/errors
9. Save code and output
10. Preserve every execution attempt
11. Edit previous practice and reload saved content
12. Upload CSV/JSON/Parquet
13. Use datasets during practice
14. Search old practices
15. Use multi-criteria filters
16. Group by date/technology/status
17. View daily/weekly/monthly reports
18. See technology-specific statistics
19. Track daily 6/6 goal and streaks
20. Run without React or another frontend server
21. Run locally through one Flask port

Instructions to Antigravity

Before writing large amounts of code, inspect the existing project directory and environment. First produce the final proposed architecture, project/file structure, database schema, dependencies, and implementation plan. Then implement the application phase-by-phase. Do not stop at architecture or mockups; continue until a functional V1 exists, running tests and fixing errors as you progress.

Priorities: 1) data/history preservation, 2) reliable execution, 3) simple local operation, 4) maintainability, 5) useful practice workflow, 6) UI polish.

Do not over-engineer this personal local application with unnecessary microservices, Docker, Kubernetes, authentication systems, React, or multiple servers.